

Hackathon Team Playbook

Ek reusable parallel-development system — Customer360 AI project pe apply kiya gaya

Yeh playbook aapke Customer360 AI pitch material se banaya gaya hai, lekin har section aise likha hai ki agle kisi bhi hackathon mein — chahe stack, team size, ya domain kuch bhi ho — reuse ho sake. Jahan bhi kuch project-specific hai, wahan **[C360]** tag laga hai. Baaki sab reusable system hai.

0. Maine aapke material se kya samjha

Product: Customer360 AI — ek Customer Decision Intelligence Platform. Sirf 'churn predict karo' nahi, balki fragmented customer signals ko ek ranked, explained, actionable recommendation mein convert karna.

Core loop (aapki hi script se): UNDERSTAND → PREDICT → EXPLAIN → DECIDE → PERSONALIZE → ACT

Pipeline (Member 3 ke part se): raw signals (transactions, clicks, carts, feedback) → RFM + engagement feature engineering → predictive layer (churn, purchase propensity, affinity, CLV) → **Next Best Action (NBA) engine** (predictions + business rules + action scoring) → SHAP-based explanation ke saath ranked recommendation.

Stated MVP stack: Python, FastAPI, PostgreSQL, React, ML services.

Team size: 4 members — Hook/Problem framing (M1), UI/UX (M2), Backend/Tech pipeline (M3), 'wow' NBA + explainability moment (M4). Yeh naturally Frontend / Backend / Data-ML / Integration-DevOps ownership pe map hota hai — jo neeche 4-member model mein hai.

Judging ke liye sabse zyada protect karne wali line: 'prediction is not the decision' — matlab NBA engine aur SHAP explanation hi product hai, koi add-on nahi. Neeche ki architecture aur task priorities is baat ko guarantee karti hain ki demo tak yeh survive kare, chahe kuch aur cut ho jaaye.

Mandatory vs optional (inferred — apne actual rubric/problem statement se confirm kar lena):

- **Mandatory:** customer signals ingest karo → features compute karo → churn/propensity/CLV predictions → NBA ranking → profile + risk + NBA card dikhane wala dashboard → har recommendation ke saath ek explanation (SHAP ya rule-based).
- **Important:** multi-channel action suggestion (WhatsApp/email/etc.), loyalty/offer logic, business users ke liye auth.
- **Optional/Future-scope:** real-time streaming signals, multi-tenant, feedback-loop learning (M4 ka 'learn from every response'), advanced RBAC.

1. Project Understanding Process

Koi bhi code likhne se pehle: poori team ke saath ek 20-30 minute ka working session, ek hi shared doc mein.

- Har source (problem statement, rubric, pitch deck) sab saath mein padho — koi akela padh ke secondhand report na kare.
- Sab kuch verbatim jahan tak ho sake, char buckets mein daalo: **Mandatory / Important / Optional / Future-scope**.
- Pehchaano: end user kaun hai (yahan: dashboard dekhne wala business user, na ki end customer), workflow kaun trigger karta hai, demo time pe screen pe 'done' kaisa dikhega.

- Har person ek line likhe: 'Judges humein isliye yaad rakhenge kyunki ____.' Agar answers alag-alag aa rahe hain, matlab alignment abhi nahi hai — step 2 dobara karo.

[C360] aapka anchor sentence hai — 'prediction is not the decision' — yaani NBA + explanation.

2. Requirement Extraction Method

Prose requirements ko atomic, testable statements mein convert karo: 'Given [state], the system shall [behavior], producing [output]'. Har ek ko M/I/O/F (Mandatory/Important/Optional/Future) tag karo. Yahi list single backlog source banegi (Section 17).

[C360] extracted mandatory requirements:

- Customer ke transaction/click/cart/feedback history se, system RFM + engagement features compute karega.
- Features se, system churn probability, purchase propensity, affinity score, aur CLV estimate output karega.
- Predictions + business rules se, system ek ranked Next Best Action output karega (product/offer, channel, timing).
- NBA output ke saath, system ek human-readable 'why' dega (SHAP values ya rule trace).
- Business user ke liye, dashboard customer profile dikhayega risk badges ke saath (jaise churn %, inactivity days, cart abandons) aur NBA card ke saath.

3. Architecture Planning

```
[Data Sources: txns, clicks, carts, feedback]
  | (seed/mock data jaldi – Section 23)
  v
[Feature Store / ETL layer] <- Data/ML owner
  | contract: feature schema (Section 9)
  v
[Prediction Services: churn | propensity | affinity | CLV] <- Data/ML
  | contract: model I/O schema (Section 12)
  v
[NBA Engine: rules + scoring + ranking] <- Backend, ML contract consume karta hai
  | contract: NBA response schema
  v
[Explainability layer: SHAP / rule-trace] <- Data/ML -> API se expose
  |
  v
[FastAPI Backend: /customers, /predictions, /nba, /explain, /auth] <- Backend
  | contract: OpenAPI spec (Section 9)
  v
[React Frontend: profile view, risk badges, NBA card, "why" panel] <- Frontend
  |
  v
[PostgreSQL: customers, events, predictions, actions, users] <- DB owner (shared schema, Section 11)
```

Dependency direction: Frontend API contract pe depend karta hai (backend implementation pe nahi). Backend DB schema + model contract pe depend karta hai (model implementation pe nahi). NBA engine prediction *contract* pe depend karta hai, kisi trained model pe nahi. Kisi ko bhi finished model file pe wait nahi karna — mock scores sabko unblock kar dete hain (Section 12).

Hour 1 se hi independently develop ho sakta hai: DB schema + seed data, API contract + mocked responses, mocked API ke against UI, fixed sample features ke against model interface, fixed mock

predictions ke against NBA ke business-rule logic.

4. Team Structure — 2 Members

Do vertical halves mein merge karo, har ek apna poora slice end-to-end own kare, sirf contract layer pe sync ho.

Member	Kya Own karega
A — Product/Frontend+Integration	React UI, API contract likhna, auth UI, demo flow, integration testing
B — Data/Backend	FastAPI, PostgreSQL, feature engineering, models (pehle simple heuristics, baad mein ML), NBA rules

Rule: A kabhi B ke real model ka wait nahi karta — B pehle hour mein agreed schema ke match karta ek mock `/predictions` endpoint fixed realistic JSON ke saath ship karta hai.

5. Team Structure — 3 Members

Member	Kya Own karega
A — Frontend + UX	React UI, dashboard, NBA card, 'why' panel, demo polish
B — Backend + DB	FastAPI, PostgreSQL schema/migrations, auth, NBA engine (rules + ranking)
C — Data/ML	Feature engineering, churn/propensity/affinity/CLV models, SHAP explainability, B/A ke integration ke liye day-1 pe mock predictions API stub

Rule: C ke mock outputs hi hour 1 se real contract hain; jab tak schema change nahi hota, C bina B ya A ka code chede mocks ko real model output se swap kar sakta hai.

6. Team Structure — 4 Members [C360 — aapki team se match karta hai]

Member	Kya Own karega	Aapke pitch roles se mapping
M1 — Backend/API + Integration lead	FastAPI endpoints, auth/RBAC, NBA rule engine, API contract doc owner, daily integration run karna	Hook/Problem framing → ab 'glue' role
M2 — Frontend/UX	React dashboard, customer profile view, risk badges, NBA card, SHAP 'why' panel rendering	UI/UX
M3 — Data/ML	Feature pipeline (RFM/engagement), churn/propensity/affinity/CLV models, SHAP integration, model contract stub	Backend + Tech
M4 — DevOps/Data/QA + Demo lead	PostgreSQL schema + seed data, environment setup, deploy pipeline, end-to-end testing, demo script + rehearsal owner	'Wow moment' / NBA reveal owner — demo ke liye bhi best fit, kyunki M4 ke paas already judges ko yaad rehne wala payload hai

Ownership feature-based hai, permanent nahi — jaise hi M4 ka seed data aur schema stable ho jaata hai (usually pehle kuch hours mein), M4 testing + deployment + demo prep mein shift kar jaata hai, DB migrations ka ownership rakhte hue.

7. Feature Decomposition Method

Har feature = ek vertical slice. Template:

```
Feature:
Frontend:
Backend/API:
Database:
Model (agar hai):
Business rules/validation:
Testing:
Integration point:
Parallelizable:
```

[C360] example — 'Customer Risk Profile' feature:

- Frontend: profile page, risk badge row (churn %, inactive days, cart abandons)
- Backend: GET /customers/{id}/profile
- DB: customers, events, predictions tables
- Model: churn + propensity scores consume honge, yahan compute nahi
- Validation: missing/partial signal data gracefully handle karo (naya customer, no history)
- Testing: /customers/{id}/profile response shape ka contract test; 0/partial data ke saath UI correctly render ho
- Parallelizable: yes, predictions schema agree hote hi — frontend turant mock ke against build kar sakta hai

8. Dependency Mapping

Har task ki real dependency list karo, phir avoidable wali ko stub se hata do.

Task	Real dependency	Avoidable?	Temporary unblock
Frontend dashboard	Real API	Haan	API contract match karti mock JSON
NBA engine	Real ML predictions	Haan	Fixed mock prediction payloads
Model training	Clean real data	Partially	Realistic distribution wala synthetic seed dataset
Auth-gated screens	Real auth backend	Haan	Fake JWT / hardcoded test user
Deployment	Sab features complete	Nahi — jaldi shuru karo	Day 1 pe skeleton app deploy karo, phir often

[C360] Critical path: DB schema → seed data → API contract → (parallel: model mock + NBA rules + mock ke against frontend) → real model swap-in → SHAP wire-in → integration testing → demo freeze.

9. Contract-First Development

Implementation code likhne se pehle define karo, ek shared file mein (jaise CONTRACTS.md ya OpenAPI spec) repo root mein:

- **API contract:** har endpoint, method, auth requirement, request schema, response schema (success + error), example payloads, status codes.
- **DB entity model:** tables, fields, types, relationships, indexes, enums.
- **Model contract:** input feature vector shape, output shape (score + confidence jahan relevant ho), error/fallback state, expected latency.
- **Auth format:** token structure, roles, protected-route list.

[C360] starter contract skeleton:

```
GET /customers/{id}/profile
-> 200 { id, name, churn_risk: float, inactive_days: int,
        abandoned_carts: int, clv_estimate: float,
        segment: string }

GET /customers/{id}/nba
-> 200 { product: string, offer: string, channel: string,
        timing: string, confidence: float,
        explanation: [{ feature: string, impact: float }] }

POST /predict/churn (internal, ML -> backend)
-> { customer_id, churn_probability, model_version }
```

Koi bhi contract change = versioned (/v1, /v2 ya ek changelog entry) aur turant team channel mein post — kabhi bhi silently edit nahi.

10. Frontend/Backend Synchronization

Backend author implementation se pehle contract likhta hai (Section 9). Frontend hour 1 se hi ek mock server (JSON fixtures, ya MSW/json-server jaisa tool) ke against build karta hai, wahi exact contract match karte hue. Jab real endpoint ready ho jaaye, frontend sirf base URL/fixture source swap karta hai — koi component rewrite nahi. Frontend kabhi field names guess nahi karta; agar koi field ambiguous hai, turant blocker raise karo, assume mat karo.

11. Database Synchronization

Schema + relationships jaldi define karo (Section 3 diagram → concrete DDL) aur Git mein versioned migrations ke through commit karo (is stack ke liye jaise Alembic), kabhi kisi ke machine pe hand-edit nahi. Data chahiye to sabko seed data pe kaam karna chahiye (Section 23), live shared dev DB pe nahi, taaki ek dusre ka kaam clobber na ho. Schema change process: CONTRACTS.md mein propose karo → quick sync message → migration file add karo → sab pull karke migrations re-run karein.

12. AI/Model Synchronization [C360-relevant]

Yeh aapka sabse high-risk integration point hai — ise explicitly protect karo.

- Model contract (Section 9) kisi bhi model train hone se pehle define hona chahiye.
- M3 day 1 pe ek **mock prediction service** ship karta hai: deterministic, hardcoded-but-plausible churn/propensity/affinity/CLV values customer ID ke against, exact response schema match karte hue.
- Backend (M1) aur frontend (M2) mock ke against integrate karte hain, model internals kabhi nahi chedte.
- Real model usi endpoint/contract ke peeche swap hota hai — agar exact contract shape hit nahi kar pata, to ek adapter mein wrap hota hai, contract change nahi hone diya jaata.
- SHAP explanation ko apna sub-contract mana jaata hai (explanation: [{feature, impact}]) taaki frontend ka 'why' panel turant mock explanation ke against ban sake, real SHAP integration ke parallel.
- Fallback: agar SHAP integration time khaane laga, to same contract satisfy karta ek rule-based 'why' ship karo (jaise 'flagged because: 74 days inactive, 2 abandoned carts') — UI ko farak nahi padta peeche kaunsa hai.

13. Authentication/RBAC Strategy

Jaldi decide karo chahe minimal ho: kaun login karega (business users), kaunse roles hain (jaise viewer vs admin, agar hai), aur backend pe enforce karo — kabhi frontend button chhupa ke rely mat karo. Hackathon ke liye, ek authenticated role hardcoded test user ke saath sabko turant unblock kar deta hai; real role checks tabhi add karo jab rubric mein chahiye ho.

14. Git/Branch/Commit Workflow

- main — hamesha demoable/working.
- dev — integration branch, stable checkpoint pe main mein merge hoti hai.
- feature/<owner>-<short-name> — har vertical slice ke liye ek short-lived branch.
- Commits: chhote, present-tense, scoped (feat(nba): add rule-based ranking, fix(api): correct churn response schema).
- PRs (informal bhi chalega, hackathon mein self-review): chhota description + kaunsa contract touch hota hai. dev mein frequently merge karo (har kuch hours mein, end mein ek baar nahi).
- Kabhi 'git add .' karke teammate ki file force-push mat karo. Shared files (requirements.txt/package.json, .env.example, schema.sql/migrations, routing config) — chedne se pehle announce karo, chhote isolated commits.
- Har session naya kaam shuru karne se pehle rebase/pull karo taaki late conflicts na aayein.

15. Local Development Synchronization

- Repo mein ek .env.example (real .env/secrets kabhi commit mat karo).
- README mein 'quickstart': clone → deps install → .env.example copy → migrations run → seed data → services start. Fresh machine pe 10 minute se kam mein chal jaana chahiye.
- Versions pin karo: Python version, Node version, Postgres version — .tool-versions/README note mein — 'works on my machine' avoid karo.
- .gitignore mein .env, node_modules, __pycache__, agar bade hain to model artifacts/checkpoints cover ho.

16. Daily Team Workflow

Chhota, blocker-focused, lambi meetings nahi:

- **Day/session shuru (10 min):** raat mein contracts mein kya change hua, har koi aage kya karega, koi blocker.
- **Midday check (5 min):** quick 'still on track / blocked' ping — async chalega (chat, call nahi).
- **Pre-integration merge (jab zaroorat ho):** dev mein merge karne se pehle quick smoke test.
- **Day/session end (15-20 min):** jo bhi mergeable hai merge karo, ek baar poora app end-to-end run karo, confirm karo demo abhi bhi chal raha hai, kal ke liye open issues log karo.

17. Task Management System

Hierarchy: **Epic** → **Feature** → **User Story** → **Technical Task** → **Subtask**. Har task card mein: owner, priority (M/I/O/F Section 2 se), dependency, expected input/output, acceptance criteria, effort estimate, status,

integration point, aur ek **BLOCKING / NON-BLOCKING** tag hona chahiye.

[C360] example Epic → Feature breakdown:

- Epic: Decision Intelligence Core
- Feature: Customer Risk Profile (BLOCKING — sab kuch downstream ke liye chahiye)
- Feature: Prediction Services (NBA ke liye BLOCKING)
- Feature: NBA Engine (demo wow-moment ke liye BLOCKING)
- Feature: Explainability Panel (MVP ke liye NON-BLOCKING, differentiation ke liye mandatory — NBA kaam karne ke turant baad prioritize karo)
- Feature: Auth (NON-BLOCKING, stub ho sakta hai)
- Feature: Deployment (day 1 se shuru, NON-BLOCKING but kabhi postpone mat karo)

18. Blocker Management — 'Never Wait' Rule

Agar kisi dusre ke incomplete kaam se blocked ho: (1) exact missing contract/interface naam do, (2) us contract ke match karta mock/stub/fixture banao ya use karo, (3) apne slice pe kaam jaari rakho, (4) shared channel mein blocker post karo — chupchaap wait mat karo.

[C360] worked examples:

- Frontend real predictions ka wait kar raha → M3 ke mock prediction JSON ke against build karo.
- NBA engine trained model ka wait kar raha → fixed mock scores ke against ranking logic build karo; baad mein swap karo.
- SHAP integration schedule se peeche → same contract ke peeche rule-based explanation ship karo, baad mein time ho to swap karo.
- Deployment 'sab complete hone' ka wait kar rahi → day 1 pe skeleton app deploy karo aur continuously redeploy karo.

19. Parallel Development Strategy

Ek baar contracts (Section 9) lock ho jaayein, chaaron members apne vertical (Sections 4-6) pe saath-saath kaam karte hain, jo bhi abhi real nahi hai uske liye mocks ke against build karte hain, aur end mein nahi balki incrementally integrate karte hain (Section 20).

20. Integration Strategy

dev mein jaldi aur baar-baar merge karo — har stable slice ke baad, deadline pe nahi. Har merge ke baad: ek smoke test run karo (app abhi bhi boot ho raha hai, primary flow abhi bhi render ho raha hai). Agar dev break ho jaaye, jisne break kiya wo turant fix kare ya revert kare; baaki log apne branches pe unaffected kaam karte rahein.

21. Testing Strategy

Demo risk ke hisaab se prioritize karo, coverage percentage ke hisaab se nahi:

- **Mandatory e2e:** exact judge-facing flow (dashboard open karo → profile dekho → NBA card dekho → 'why' dekho) seed data ke saath har baar kaam kare.
- **API contract tests:** har endpoint agreed shape return kare.

- **Business rule unit tests:** NBA ranking logic, churn threshold logic.
- **DB consistency:** seed data cleanly load ho, relationships hold ho.
- Time pressure mein exhaustive frontend/unit coverage skip karo — pehle demo path protect karo.

22. DevOps/Deployment Strategy

Day 1 se shuru karo: repo scaffold ho, install script kaam kare, agar time ho to basic CI (push pe 'install + run tests' bhi chalega), ek environment jahan koi bhi deploy kar sake aur judges dekh sakein. Shuru se hi health check endpoint (/health). Har stable integration merge ke baad deploy karo, end mein ek baar nahi — deployment ko last-hour surprise mat banne do. **[C360]:** FastAPI + Postgres + React ko jaldi containerize karo (simple Docker Compose bhi chalega) taaki model service ko baaki stack chede bina swap kiya ja sake.

23. Data/Seed Strategy

Real data pipelines banne se pehle realistic seed data define karo: consistent IDs wale sample customers ka ek fixed set, varied churn/propensity/CLV values (edge cases sahit — naya customer, no history, high-value at-risk customer jo aapki demo narrative match kare), taaki har member ek hi known-good dataset ke against kaam kare. Test/demo runs ke beech seed data cleanly reload karne ke liye reset script. Random/live-mutated data ke against kabhi demo mat karo.

24. Change & Scope Management

Hackathon ke beech mein naya idea aaye → accept karne se pehle Critical / Important / Optional / Post-hackathon classify karo (Section 2 tags). Critical path (Section 8) protected hai — kuch bhi add nahi hota jo mandatory demo flow ko threaten kare. Accept karne se pehle: frontend/backend/DB/model/testing/demo pe impact jaldi assess karo; agar koi owner kahe 'yeh mera contract break karta hai', to wo wait karega.

25. Risk Management

Risk	Preventive strategy	Fallback
Backend delays	Contract-first + hour 1 se mocks	Frontend mock ke against demo karta rahega agar real API slip ho
Model instability/kam accuracy	Same contract ke peeche rule-based fallback	Honestly 'MVP heuristic, real model in progress' present karo
DB migration conflicts	Versioned migrations, har schema change ka ek owner	Corrupt ho to seed script se rebuild karo
API contract late change	Contracts version karo, turant announce karo	Consumers rewrite karne ki jagah adapter layer
Auth complexity time kha rahi	Auth jaldi stub karo, zaroorat pe hi real checks	Single test user ke saath ship karo, known limitation note karo
Deployment failure deadline ke paas	Day 1 se continuously deploy karo	Local demo fallback, recorded video backup
Demo data missing/broken	Fixed seed dataset, reset script	Final freeze se pehle seed data reload karo
Last-minute UI changes	Feature-freeze window (Section 26)	Last stable main pe revert karo

26. Hackathon Time Management (apni actual duration ke hisaab se ratios adjust karo)

Stage 1 - Samajhna + Architecture + Contracts	(~10-15%)
Stage 2 - Foundation (schema, seed data, scaffolding, mocks)	(~10%)
Stage 3 - Parallel implementation + continuous integration	(~45-50%)
Stage 4 - Feature completion + real-service swap-in	(~15%)
Stage 5 - Testing + polish	(~10%)
Stage 6 - Demo prep + final freeze	(~5-10%)

Koi exact hours assume nahi kiye gaye — jitna bhi window mila hai usi ke hisaab se har stage proportionally scale karo, aur Stage 6 se pehle ek hard **feature freeze** set karo taaki time ka aakhri block naya code nahi, testing/rehearsal ho.

27. Demo Preparation [C360]

Aapki apni script ek strong asset hai — ise sirf pitch nahi, demo spec maano:

- Judges ko dikhne wala exact click-path rehearse karo: dashboard → risk badges wala customer profile → NBA card reveal ('BUT... NOW WHAT?') → SHAP/why panel.
- 2-3 pre-verified seed customers rakho, flagship wala bhi (high-value, 82% churn, 74 days inactive, 2 abandoned carts) taaki screen pe numbers aur story hamesha match karein.
- Ek person (M4, demo lead) rehearsal timing owns kare aur 'agar break ho jaaye to kya' fallback (local build + recorded backup clip).
- Present karne se kam se kam ek poore test cycle pehle UI aur demo data freeze karo — freeze ke baad koi live edit nahi.

28. Final Quality Gate ('Definition of Done' vs 'Demo Ready')

Definition of Done ek feature ke liye: UI + API + business logic + DB persistence + validation + error/empty/loading states + integration + basic tests sab saath kaam kar rahe — sirf 'UI exist karti hai' kaafi nahi.

Demo Ready: exact demo click-path ke under, seed data ke saath, do baar consecutively stable, koi console error nahi, jis machine pe actually present karoge usi pe.

29. Universal Checklist

- Sab mandatory requirements (Section 2) implement aur demonstrable hain
- Primary user flow seed data ke saath end-to-end chalta hai, do baar consecutively
- Sab API responses committed contract match karte hain
- DB migrations fresh clone pe cleanly run hote hain
- Auth/authorization server-side enforce ho raha, sirf UI mein hidden nahi
- Error/empty/loading states handle hain, sirf happy path nahi
- Koi secret commit nahi hua; .env.example present aur accurate hai
- Deployment fresh environment se kaam karta hai, sirf localhost se nahi
- README quickstart ek aisi machine pe verify hua jo original dev ki nahi hai
- Demo script frozen seed data ke against rehearse hua hai
- Known limitations honestly document hui hain (judges ko live discover na karna pade)

- Judging criteria screen pe jo dikh raha hai usse explicitly map hui hai

30. Worked Example — Ek Feature, Teen Team Sizes

Feature: 'Churn %, inactivity, aur NBA card ke saath customer risk profile dikhana'

2 members:

A (Product/Frontend+Integration) API contract likhta hai, profile UI + NBA card mock JSON ke against turant build karta hai. B (Data/Backend) parallel mein DB schema, seed data, aur A ke contract match karta mock /profile + /nba endpoint banata hai — kisi bhi taraf wait nahi. B baad mein bina A ka code chede mock scores ko real heuristics/model se swap karta hai.

3 members:

A (Frontend) hour 1 se mock ke against UI banata hai. B (Backend+DB) A ke parallel, agreed contract ke against schema, seed data, aur real /profile/nba endpoints banata hai. C (Data/ML) feature engineering + churn/propensity/CLV models banata hai ek mock-then-real prediction endpoint ke peeche jo B ka NBA logic consume karta hai — C aur B pehle prediction contract agree karte hain, phir independently kaam karte hain.

4 members [C360]:

M1 (Backend/API) /profile aur /nba contracts define aur own karta hai, M3 ke mock predictions ke against NBA rule/ranking engine banata hai. M2 (Frontend) M1 ke mocked contract ke against hour 1 se profile screen + risk badges + NBA card + 'why' panel banata hai, M1 ke real API live confirm karte hi swap kar leta hai. M3 (Data/ML) RFM features, churn/propensity/CLV models, aur SHAP explanations banata hai agreed prediction/explanation contract ke peeche — turant ek mock version ship karta hai taaki M1 aur M2 kabhi wait na karein. M4 (DevOps/DB/QA) pehle schema + seed data banata hai (B/C ke real implementations unblock karte hue), phir deployment, end-to-end testing mein shift hota hai, aur pipeline stabilize hote hi demo rehearsal own karta hai.

Chaaron members same feature pe ek saath kaam karte hain, sirf shuru ke 20-minute contract agreement ke alawa kisi cheez se blocked nahi — kyunki har downstream piece kisi dusre ke finished code ke against nahi, balki contract match karti mock ke against banaya gaya tha.